

Laboratorium 9: Organizacja pamięci, adresacja i wskaźniki

Opracowanie zestawu zadań (część teoretyczna)

Kacper Bednarczuk

Zadanie 1.1: Reprezentacja zmiennych w pamięci (Endianness)

Analizowana zmienna typu `int` ma wartość szesnastkową `0x12345678`.
W architekturze **Little-Endian** (standard Intel/AMD), bajty zapisywane są w kolejności od najmniej znaczącego (LSB) do najbardziej znaczącego (MSB).

Dla liczby `0x12345678`:

- Najmniej znaczący bajt (LSB): `0x78`
- Najbardziej znaczący bajt (MSB): `0x12`

Tabela zawartości pamięci (Little-Endian):

Adres pamięci	Wartość (HEX)	Opis
<code>0x00FF0000</code>	78	Bajt najmłodszy (LSB)
<code>0x00FF0001</code>	56	
<code>0x00FF0002</code>	34	
<code>0x00FF0003</code>	12	Bajt najstarszy (MSB)

Pytanie dodatkowe (Big-Endian):

Gdyby procesor działał w trybie **Big-Endian** (np. procesory Motorola, protokoły sieciowe TCP/IP), kolejność bajtów byłaby zgodna z naturalnym zapisem liczby (od MSB do LSB).
Pod adresem bazowym `0x00FF0000` znalazłaby się wartość `0x12`, a pod `0x00FF0003` wartość `0x78`.

Zadanie 1.2: Wyrównanie pamięci (Padding)

Analizowana struktura:

```
struct Student {  
    char inicjal; // 1 bajt
```

```
int numer_id; // 4 bajty
short wiek;    // 2 bajty
}
```

1. Rozmiar teoretyczny

Suma rozmiarów poszczególnych pól bez uwzględnienia wyrównania:

$$1\text{ B} + 4\text{ B} + 2\text{ B} = 7\text{ bajtów}$$

2. Rozmiar rzeczywisty

Przy typowym wyrównaniu (alignment) do 4 bajtów, kompilator wstawia puste bajty (*padding*), aby zmienna `int` trafiła pod adres podzielny przez 4, oraz aby cała struktura miała rozmiar będący wielokrotnością największego pola.

- `char` (offset 0) zajmuje 1 bajt.
- `int` wymaga wyrównania do 4 bajtów, więc następuje przesunięcie do offsetu 4 (3 bajty paddingu).
- `short` trafia na offset 8.
- Cała struktura kończy się na offsecie 10, ale jej rozmiar musi być podzielny przez 4. Dodawane są 2 bajty końcowego paddingu (*tail padding*).

Całkowity rozmiar: **12 bajtów**.

3. Układ pamięci (Mapa)

Offset	Zawartość	Rozmiar	Komentarz
0	char inicjal	1 B	
1-3	<i>padding</i>	3 B	Wyrównanie dla pola <code>int</code>
4-7	int numer_id	4 B	
8-9	short wiek	2 B	
10-11	<i>padding</i>	2 B	Wyrównanie całkowite struktury

Zadanie 1.3: Segmentacja pamięci procesu

Przyporządkowanie elementów programu do segmentów pamięci:

Element programu	Segment pamięci	Uzasadnienie
A. Zmienna lokalna <code>int x = 5;</code>	3. Stos (Stack)	Zmienne lokalne istnieją tylko w obrębie wywołania funkcji.
B. Kod maszynowy funkcji	1. Segment Kodu (Text)	Instrukcje procesora są niezmiennie i tylko do odczytu.

Element programu	Segment pamięci	Uzasadnienie
C. Zmienna dynamiczna (<code>new</code> / <code>malloc</code>)	4. Sterta (Heap)	Pamięć alokowana ręcznie w czasie działania programu.
D. Zmienna globalna <code>int g_licznik = 0;</code>	2. Segment Danych (Data)	Zmienne dostępne przez cały czas życia procesu.
E. Stała tekstowa <code>"Witaj świecie"</code>	2. Segment Danych (ROData)	Literały tekstowe są przechowywane jako dane tylko do odczytu.

Zadanie 1.4: Arytmetyka wskaźników

Wskaźnik `int* ptr` posiada wartość adresu bazowego `0x1000`.

W arytmetyce wskaźników dodanie liczby całkowitej N powoduje przesunięcie o $N \cdot \text{sizeof}(\text{typ})$.

Obliczenia:

a) `ptr + 1`

Przesunięcie o jeden element typu `int` (4 bajty).

$$0x1000 + 1 \cdot 4 = 0x1000 + 0x4 = \mathbf{0x1004}$$

b) `(char*)ptr + 1`

Rzutowanie na `char*` sprawia, że krok wynosi 1 bajt (rozmiar `char`).

$$0x1000 + 1 \cdot 1 = 0x1000 + 0x1 = \mathbf{0x1001}$$

c) `ptr + 4`

Przesunięcie o cztery elementy typu `int` (łącznie 16 bajtów).

$$0x1000 + 4 \cdot 4 = 0x1000 + 16 = 0x1000 + 0x10 = \mathbf{0x1010}$$